TERRAFORM PLUM

DevOps & Cloud [SWE] — Class 19 Notes

LECTURE TITLE	Terraform & Infrastructure as Code — the five blocks, the ten commands, and a live S3 bucket on a real AWS account
DATE/TIME	29 September 2026, 11:15 AM
DURATION	120 minutes
INSTRUCTOR	Nensi Ravaliya
BIG IDEA	Stop clicking. Write down *what* you want in a `.tf` file (a provider, some variables, a resource, an output), and let `terraform plan` show you the change and `terraform apply` make it real — the same way every time, in every environment.
SOURCES	class transcript + her repo folder for this session, `Nency-Ravaliya/devops-heros` → `session18-terraform-iac` (`01-iac-basics` … `09-state` and `terraform-s3-demo`) + your own working files in this folder (`03-providers/main.tf`, `terraform-s3-demo/*.tf`, the state files, and the two screenshots).

WHAT'S INSIDE

0 ⚡ The 60-Second Version
1 🗓️ Logistics — Credentials, the Task Sheet & the Final Project
2 🖱️ The Manual Way — Launching an EC2 Instance by Clicking
3 🏗️ What Terraform Is — and Why Clicking Doesn't Scale
4 ⚖️ "Why Not Just Use Helm?" — Provisioner vs Orchestrator
5 🌍 Multi-Cloud & the Single Point of Failure
6 🛠️ Setup — Terraform CLI + AWS CLI
7 📄 Terraform Files & the Language
8 🧱 Block 1 — `terraform {}`: Versions and Providers
9 🔌 Block 2 — `provider`: Which Cloud, Which Region
10 🔤 Block 3 — `variable`: Inputs
11 🪣 Block 4 — `resource`: The Thing You're Creating
12 📤 Block 5 — `output`: Did It Work? What Did It Make?
13 🧩 The Whole File, Together
14 ⌨️ The Commands — `init` to `destroy`
15 ☠️ `terraform destroy` — The Dangerous One
16 🔑 IAM Live — Making the Student User and Configuring the CLI
17 🚀 The Live Demo — `terraform-s3-demo`
18 🧐 Reviewing Your Own Files
19 🏋️ Practice Labs
20 🔭 Next Class Preview — A Whole VPC
21 🎯 Quiz & Q&A Recap
22 📝 Homework
23 📋 Cheat-Sheet
24 🗒️ Notes on the Transcript Itself

NOTE

The title is right this time. After three classes in a row where the folder name lied (Classes 12, 13 and 14), this one does what it says: **Terraform and Infrastructure as Code**. In her repo it is `session18-terraform-iac` — her session numbers don't match the folder numbers, and that's expected. This is part 1 of 2: the next class (*Cloud & Terraform in Action*) builds a whole VPC.

NOTE

About the credentials. For the hands-on part she created a temporary AWS IAM user, shared its access keys with the batch for about 10 minutes, and revoked them after class. **No keys, account IDs or secrets appear anywhere in these notes** — and they shouldn't appear in yours either (see §16 and §18).

---

0 ⚡ The 60-Second Version

1. **Provisioning = creating infrastructure in the cloud.** Clicking through the AWS console to launch an EC2 instance works once. Doing it for dev, stage *and* prod, with different sizes each time, is where people make mistakes.
2. **Terraform (by HashiCorp) is an *infrastructure provisioner*.** You describe the infrastructure in `.tf` files, and one command creates it. Kubernetes/Helm is an *orchestrator* — it runs your app *on* infrastructure that something else (Terraform) created.
3. **Everything in Terraform is a block.** Learn five: `terraform {}` (versions), `provider` (which cloud, which region), `variable` (inputs), `resource` (the thing to create) and `output` (what to print back).
4. **You don't memorise keywords.** `aws_s3_bucket`, `bucket_prefix`, `id` are all predefined by the provider. Copy them from the official HashiCorp docs and just fill in *your* values and names.
5. **The commands, in order:** `init` → `fmt` → `validate` → `plan` → `apply` → `output` → `state list` / `state show` → `plan -destroy` → `destroy`.
6. **`plan` is a preview; `apply` does it for real; `destroy` deletes everything Terraform made** — in seconds, with no undo. That's why nobody gets full Terraform access on day one.
7. **Live demo:** she made an IAM user with only S3 permissions, the class ran `aws configure`, then `terraform-s3-demo` — and hit every real-world error: bad credentials, access denied, and "*bucket name not available*" (S3 names are global). About 25 buckets got created. Yours was `anishka2307`, created and then destroyed cleanly.
8. **Homework:** five READMEs on five AWS services (EC2, S3, VPC, IAM, DynamoDB/RDS) — she'll quiz you for 10 minutes at the start of the next class.

---

1 🗒️ Logistics — Credentials, the Task Sheet & the Final Project

No recap today — "*we are already running out of our time.*" She opened with `git pull` on the DevOps Heroes repo and went straight to `session18-terraform-iac`.

The credentials deal.

> SAID IN CLASS
>
> *"I will share credentials with you, but I will give it to you for 10 minutes… I will delete. You need to perform the task in like 10 minutes."*

> CLASS MOMENT
>
> **You were the lucky batch.** *"Yesterday in section B they did not get these credentials, but somehow we managed to get the credentials today."*

Because nobody in the class has their own AWS access, **both Terraform sessions (today and the next) are do-it-in-class sessions**: your assignment *is* the in-class work. The rule is:

- Do every cloud step while the keys are live.
- **Screenshot everything** — once access is revoked, you can't redo it.
- *"Don't tell me that you can't put the screenshot in the README. That is your problem."* Save the images in the class folder (which is exactly what your `Readme.MD` + `image.png` + `image-1.png` do — see §17).

The task sheet. She updated the tasks and assignments for **all 21 sessions** — including the ones already done — in a sheet linked from the root `README.md` of DevOps Heroes. Go and check what each one asks you to submit.

The final DevOps project. The last session is for this:

What	Detail
The app	A **full-stack** application — frontend, backend and a database (**Postgres or MongoDB**) — each containerised
The stack	Everything from the course: **Docker, Kubernetes, CI/CD, Terraform**, then **monitoring**
Language	She has a Java version and a Python version. The class voted **Python**, so she'll demo the Python one; you build yours (you may adapt her project)
Instructions	Grading components, what to submit, how and when — *"everything I will mention in README.md"*. Deadline set by the program team (maybe next week)
When	The last session is **online, Sunday 11 AM, both batches together**

> SAID IN CLASS
>
> *"After that don't ping on WhatsApp or everywhere like 'ma'am, I don't know what to do in this project'. Everything I will mention in the README."*

Three things to submit before the exam: **the 21 session assignments**, **the final project**, and whatever the last online session asks for. For the project's Terraform part, since you won't have credentials, she said you may just write the Terraform files even if you can't actually create the full infrastructure — details to come.

Timing for today: fundamentals until **12:40**, credentials shared at 12:40, task done by about **12:50**, plus a 10-minute buffer to explore.

---

2 The Manual Way — Launching an EC2 Instance by Clicking

Before Terraform, she showed what Terraform replaces. "*Tell me what you need to do to create an EC2 instance.*" The class: first you need a login.

WATCH IT HAPPEN

Launching an EC2 instance from the console, click by click.

```
Frame 1  Log in to the AWS console (username + password, maybe MFA)
Frame 2  Search "EC2"  →  Launch instance
Frame 3  Name it            e.g.  yatri-nancy
Frame 4  Pick an AMI        Ubuntu → choose a version (22, 24, 26 …)
Frame 5  Pick instance type micro / medium / large / xlarge …
Frame 6  Key pair           none yet → "Create new key pair" → download the .pem file
Frame 7  Network rules      allow SSH (22), HTTP (80), HTTPS (443)
Frame 8  Launch instance    → wait → running
```

Every one of those choices — name, image, size, key, firewall ports — is just a **value**. Terraform's whole trick:

SAID IN CLASS

"Whatever I did from here, manual steps, I just put it inside one file — a Terraform file — and Terraform will automatically create it for me, if I give my credentials to Terraform."

PICTURE THIS

Words from that walkthrough.

Term	Plain meaning
AMI (Amazon Machine Image)	The operating-system "install disc" the server boots from — Ubuntu 24, Ubuntu 26, Amazon Linux…
Instance type	The server's size: how many vCPUs and how much RAM. `micro` is small and cheap, `xlarge` is big
Key pair (`.pem`)	A lock-and-key for SSH. AWS keeps the lock (public key), you download the key (private `.pem`). Lose it and you can't SSH in
Security group rules	The server's firewall — which ports are open to whom

NOTE

Filled in: the ports. The audio has "*20 22*" for SSH and "*4.4.3*" for HTTPS. The standard values are **SSH = 22, HTTP = 80, HTTPS = 443**. She also described the instance as "*2 vCPU, 0.5 GiB memory*" and picked "micro" — on AWS, 2 vCPU / 0.5 GiB is actually `t3.nano` ; `t3.micro` is 2 vCPU / 1 GiB. The exact numbers don't matter for this class; the point is "pick a size per use case."

2.1 Permissions — who is allowed to do what

She then explained what "sharing credentials" really means:

> SAID IN CLASS
>
> *"Let's suppose I give the permission to create EC2 instance… If I did not give you the permission to delete EC2 instance, then you are not able to delete any EC2 instance… We call it as a role-based access control."*

So access to a cloud account is never all-or-nothing. You get an identity, and that identity carries a list of allowed actions.

> CLARIFIED
>
> **Clarified, since AWS uses "role" for something specific.** Her idea — *give someone only the permissions they need* — is exactly right, and "role-based access control" is the right general name for it. But in AWS, what she actually did in §16 was **IAM user + IAM group + an attached policy** (`AmazonS3FullAccess`). An **IAM role** in AWS is a different thing: an identity *without* permanent keys that a service or person temporarily "puts on" (e.g. an EC2 instance assuming a role to read S3). For her course, "RBAC = give each user only the permissions they need" is the answer to give.

---

3 🏗️ What Terraform Is — and Why Clicking Doesn't Scale

> LOCK THIS IN
>
> **Terraform is an infrastructure provisioning tool built by HashiCorp.** You write the infrastructure as code; Terraform creates it in the cloud. That is **Infrastructure as Code (IaC)**.

3.1 The three-environments problem

Her scenario: a website that needs four or five services.

Need	AWS service
Compute (run the app)	**EC2** — or an **EKS** cluster
Storage	**S3** bucket
Database	**RDS** (relational)
Access control	**IAM**

Now you need that **in dev, in stage and in prod**. Manually, that's three full rounds of clicking. With Terraform:

> SAID IN CLASS
>
> *"I don't need to go to my AWS console… I just need to create one file, a Terraform file, where I mention my resource — how much CPU, memory, which operating system — and then I will just run that file… It is just one command."*

The second problem is the twist: the environments are *the same shape* but *different sizes*.

```
                same code, same services
        ┌──────────┬──────────┬──────────┐
        │   dev    │  stage   │   prod   │
        ├──────────┼──────────┼──────────┤
EC2     │ 1 CPU-ish│  2× CPU  │  4× CPU  │
S3      │10 buckets│10 buckets│30 buckets│
        └──────────┴──────────┴──────────┘
                only the NUMBERS change
```

SAID IN CLASS

"Every time my resource count will change if my environment is changing… we don't want to deploy it manually one by one each environment. That's why we need Terraform."

That's exactly what **variables** are for (§10): same file, different values per environment.

PICTURE THIS

The tea-recipe analogy again. A Dockerfile was the recipe for an *image*. A `.tf` file is the recipe for the *kitchen itself* — the stove, the fridge, the gas connection. Cook the recipe in three kitchens (dev/stage/prod) and you get three identical kitchens; change "2 burners" to "4 burners" and only prod gets the bigger stove.

GOOD TO KNOW

Why IaC wins (from her repo's `01-iac-basics/README.md`): repeatability · automation · version control · consistency · review through Git (pull requests on infrastructure!) · easier disaster recovery · easy CI/CD integration.

3.2 It's small — on purpose

SAID IN CLASS

"There are hardly five commands in Terraform. It's very, very easy… you don't need to remember how to write a complete file. You just need to remember which HashiCorp Terraform version you need, and which cloud platform you are using."

Modules exist — reusable packages of Terraform code — but she flagged them as advanced and out of scope. "*Right now in this lecture you're going to learn the fundamental thing of Terraform,*" the same way CI/CD started with *why → runners → secrets → variables*.

3.3 In real life it runs in a pipeline

Today you'll type the commands by hand, but in a company they run inside CI/CD — connecting straight back to Classes 17–18:

SAID IN CLASS

"In CI/CD pipeline we are selecting one runner — Ubuntu latest… In that server we need to install all the dependencies. We installed Python, we did `pip install -r requirements.txt`… Same thing, this is Terraform. First you need to install the Terraform version, second, which cloud you are using your Terraform for."

```
GitHub Actions runner (ubuntu-latest)
   ├─ install Terraform CLI          ← like installing Python
   ├─ terraform init                 ← like pip install (downloads the AWS provider)
   ├─ terraform plan                 ← like running tests: show what would change
   └─ terraform apply                ← like deploy
```

4 ⚖️ "Why Not Just Use Helm?" — Provisioner vs Orchestrator

A student asked: *why not just use Helm instead of Terraform?* And then: *is Kubernetes a sub-part of Terraform?* Her answer: **no — they're completely different tools.**

SAID IN CLASS

"Kubernetes is an orchestrator. And Terraform is an infrastructure provisioner."

PICTURE THIS

Building vs running the restaurant.

- **Terraform** builds the restaurant: the land (region), the walls and doors (VPC, security groups), the gate to the street (internet gateway), the store-room (S3), the records office (RDS/DynamoDB), and the kitchen itself (the **EKS cluster**).
- **Kubernetes + Helm** run the kitchen once it exists: which cooks (pods) are on shift, restarting the ones who collapse, adding cooks at rush hour.

```
Terraform creates ──►  VPC · subnets · IGW · security groups · S3 · RDS · IAM · EKS
                                                                          │
Helm/Kubernetes deploys ──────────────────────────────────────────────►  your pods
```

SAID IN CLASS

"EKS and Helm is only one service, only one part… With this EKS you need internet gateway, with that you need security groups, availability zones, regions, storage, maybe DynamoDB or RDS. All these services you need."

SAID IN CLASS

(Hindi, translated) "*Kubernetes manages your containers and your workload once they're deployed. But the EKS service itself — if you'd made it by hand, Terraform is what makes it.*"

She also gave the two database families as an aside:

Type	AWS service
Relational (SQL, tables)	**RDS**
Non-relational (NoSQL)	**DynamoDB**

And one microservice in a real company can need **10, 20, 30, 40+** services — that's the scale where provisioning by hand stops being possible.

---

5 🌍 Multi-Cloud & the Single Point of Failure

The `required_providers` block (§8) lets you list **more than one** cloud — AWS *and* Azure *and* GCP in the same project. That's **multi-cloud**.

Her story (half in Hindi):

SAID IN CLASS

"*Suppose one AWS region in the whole world goes down, and you deployed your service in that AWS region. AWS will pay you for however much downtime there was — but what about your users?*"

WATCH IT HAPPEN

One region dies.

```
Frame 1  Your shop runs only in AWS us-east-1.  Competitors too.
Frame 2  us-east-1 goes down.  Everyone's site is down.
Frame 3  Single-cloud you: down with everyone else. AWS credits don't bring customers back.
Frame 4  Multi-cloud you: Azure copy keeps serving.
Frame 5  Every frustrated shopper from the dead sites lands on yours. You make MORE money.
```

LOCK THIS IN

The main reason for multi-cloud is avoiding a single point of failure — one region or one provider going down shouldn't take you down. The extra business is the indirect bonus. "*It's a very good practice… more of the companies are doing the same thing. MNC companies do this.*"

GOOD TO KNOW

She has public repos with a **multi-cloud Terraform three-tier project** (AWS + Azure, a Java full-stack app) on her GitHub for anyone who wants to go further.

6 🛠️ Setup — Terraform CLI + AWS CLI

You need **two** tools. The links are in the root `Readme.md` of `session18-terraform-iac`:

Tool	Link (from her repo)	Check it works
Terraform CLI	`developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli`	`terraform version`
AWS CLI	`docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html`	`aws --version` (or just `aws` → you'll see the help/usage text)
Terraform + AWS tutorial	`developer.hashicorp.com/terraform/tutorials/aws-get-started/aws-create`	—

SAID IN CLASS

"I don't know specific command, maybe AWS CLI version… But once you type `aws` you will see AWS help, AWS command help kind of a thing. That means your AWS CLI is installed." — the actual flag is `aws --version`.

DO THIS YOURSELF

Lab 1 — Install and verify (10 minutes, no AWS account needed).

1. Install Terraform. On macOS: `bash brew tap hashicorp/tap brew install hashicorp/tap/terraform`
2. Install the AWS CLI (macOS: `brew install awscli`, or the `.pkg` from the AWS link).
3. Verify both: `bash terraform version # e.g. Terraform v1.16.x on darwin_arm64 aws --version # e.g. aws-cli/2.x.x Python/3.x Darwin/...`
4. Only when you have keys: `aws configure` (see §16), then prove who you are: `bash aws sts get-caller-identity` It prints your user's ARN — if it errors, Terraform will error too.

NOTE

Your state file records **Terraform 1.16.4** and your lock file pins the **AWS provider 6.66.0** — so on 29 September you were well above the `>= 1.6.0` minimum and inside `~> 6.0`.

7 Terraform Files & the Language

7.1 One file type

SAID IN CLASS

"For Docker we have one type of file — Dockerfile. Same way here we have a Terraform file. Every file that we are going to write is a `.tf` file."

File	What it's for
`*.tf`	Terraform code. Any name works; Terraform reads **every** `.tf` file in the folder as one big configuration
`main.tf`	The conventional "default" file — where you start, and where today's concept walkthrough lived
`*.tfvars` (e.g. `terraform.tfvars`)	**Values** for your variables — per environment, and often secret, so it's git-ignored
`terraform.tfstate`	Created *by* Terraform after `apply` — its memory of what it built (§14.6)
`.terraform/`	Created by `init` — downloaded provider plugins
`.terraform.lock.hcl`	Created by `init` — the exact provider version picked

7.2 Split by job (the standard names)

She taught every block inside one `main.tf`, then showed the demo folder where each block lives in its own file:

```
terraform-s3-demo/
├── terraform.tf      ← terraform {} block   (versions + required providers)
├── providers.tf      ← provider "aws" {}    (region)
├── variables.tf      ← variable {} blocks   (inputs)
├── main.tf           ← resource {} blocks   (the actual infrastructure)
├── outputs.tf        ← output {} blocks     (what to print back)
└── .gitignore        ← keeps .terraform/, *.tfstate, *.tfvars out of Git
```

SAID IN CLASS

"Let's suppose you have three providers — then you need to mention three blocks inside the `provider.tf` file. That's why I am segregating every file."

7.3 Don't memorise — copy from the docs

SAID IN CLASS

"No one will ask you in the interview 'write a Terraform file' by yourself. Concept — what does it mean by variable, how to use a variable."

Every resource type and argument name is **predefined** by the provider. The official docs at **developer.hashicorp.com** have Terraform guides for AWS, Azure, Docker, GCP, HCP Terraform, OCI — and for anything not listed (her example: **Salesforce**) you just search "*Terraform Salesforce*" and that vendor's own docs have the provider.

The catch: keywords change per cloud. `aws_s3_bucket` is AWS's storage keyword; Azure's storage has a *different* keyword. So the only things **you** invent are:

1. the **local name** you give each resource (e.g. `provider_demo`), and
2. the **values** (bucket name, region, tags, CPU, …).

NOTE

Filled in: she asked "*anyone knows the storage that we are using inside Azure?*" and no answer is audible. It's **Azure Blob Storage**, inside a **Storage Account** — Terraform resource `azurerm_storage_account` (provider `hashicorp/azurerm`).

7.4 The syntax

SAID IN CLASS

"*Everything inside Terraform — curly braces… If you want to use this code somewhere else you just copy and paste it. It's not dependency, just curly braces.*"

CLARIFIED

Clarified: it looks like JSON, but it's HCL. She called the curly-brace format "JSON format". The language is actually **HCL — HashiCorp Configuration Language**. It *resembles* JSON (braces, key = value) but uses `=` instead of `:`, needs no commas between lines, and allows comments (`#` or `//`). Terraform *also* accepts pure JSON in files named `*.tf.json`, which is probably where the mix-up comes from. Her real point — *each block is self-contained, so you can copy-paste blocks between projects* — is spot on.

LOCK THIS IN

The universal block shape:

```
<BLOCK_TYPE> "<LABEL_1>" "<LABEL_2>" {
  argument = value
}
```

"*Everything in Terraform is a block.*"

8 Block 1 — `terraform {}`: Versions and Providers

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

8.1 `required_version`

This is the Terraform **CLI** version the project needs. She called it a concept "*most people who know Terraform for 5 years also don't know*": why `>=` ?

> SAID IN CLASS
>
> "*Bare minimum you need 1.6.0 version to run your actual project. For each project this will change.*"

- **Why a minimum at all?** New features land in new versions; an older CLI can fail to understand newer code.
- **Why `>=` and not an exact pin?** So newer CLIs are still allowed. "*If you want a precise version you can mention directly the version. But for best practice… greater than or equals to.*"

Her explanation of what happens on a mismatch:

> SAID IN CLASS
>
> "*If I have a 1.5.0 version installed in my local, whenever I am doing the `terraform init` it will upgrade my Terraform version to 1.6.0. If I already have 1.6.0, then it will check… is it satisfying the condition? Yeah, so it will not update.*"

> CLARIFIED
>
> **Clarified — Terraform never upgrades itself.** `required_version` is a **check**, not an installer. If your CLI is 1.5.0 and the file says `>= 1.6.0`, `terraform init` (and every other command) **stops with an error** — "*Unsupported Terraform Core version… This configuration does not support Terraform version 1.5.0*" — and **you** upgrade the CLI (`brew upgrade terraform`, or a version manager like `tfenv`). The *purpose* she described is right (guarantee a minimum version); only the "it upgrades for you" part isn't. In her assessment, "it ensures the minimum Terraform version the project needs" is the safe answer.

8.2 `required_providers`

> SAID IN CLASS
>
> "*How many providers — cloud providers — you need to use to deploy your application… If you have multiple providers, you just copy this tiny block and paste it, and my provider is Azure.*"

Part	Meaning
`aws`	The local name you'll use for this provider
`source = "hashicorp/aws"`	Where to download it from: the Terraform Registry, namespace `hashicorp`, type `aws`
`version = "~> 6.0"`	Which provider versions are allowed

CLARIFIED

Clarified — `required_providers` installs a *plugin*, not the AWS CLI. She described it as "*it will install the version for your AWS CLI*" and "*for Azure there is one Azure CLI that you need to install.*" What `terraform init` actually downloads is the **AWS provider plugin** (a Terraform add-on, into `.terraform/`) — the piece that knows how to talk to AWS's API. The **AWS CLI** is a separate tool you installed yourself in §6; Terraform doesn't need it to run, but it's handy because `aws configure` writes the credentials file that the provider reads.

WHY IT WORKS

What `~> 6.0` means (the "pessimistic" operator). It allows the *last* number written to go up, nothing to its left:

Constraint	Allows	Blocks
`~> 6.0`	6.0, 6.1 … **6.66** … 6.99	7.0 and above
`~> 6.66.0`	6.66.0, 6.66.1 …	6.67
`>= 1.6.0`	1.6.0 and anything newer, forever	older

Major versions can contain breaking changes, so `~>` lets you get fixes without surprises. Your `.terraform.lock.hcl` shows `init` picked **6.66.0** under `~> 6.0`.

9 🔌 Block 2 — `provider`: Which Cloud, Which Region

```
provider "aws" {
  region = var.aws_region
}
```

SAID IN CLASS

"To mention the provider it's very easy — provider, and then provider name. And inside curly braces, which region you want to use. Let's suppose I want to use an India region, where we have ap-south-1, Bombay region."

- `"aws"` must match the name in `required_providers`.
- `region` here is `var.aws_region` — not a hard-coded string, but a reference to a variable (next section). `var.` means "go and get the variable called…".

WATCH OUT

Never put keys in the provider block. Her repo's `03-providers/README.md` spells it out:

```
provider "aws" {
  access_key = "..."   # ❌ NO
  secret_key = "..."   # ❌ NO
}
```

The moment that file is committed, your keys are on GitHub. Use `aws configure` (which saves them in `~/.aws/credentials`, outside your repo) and let the provider pick them up automatically.

WHY IT WORKS

Where the provider gets credentials from (in order of what you used today). You never typed a key into a `.tf` file — so how did Terraform log in? The AWS provider looks in the standard places: environment variables (`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`), then the shared files `~/.aws/credentials` + `~/.aws/config` that `aws configure` wrote. That's the bridge between §16 and §17.

---

10 🔤 Block 3 — `variable`: Inputs

```
variable "aws_region" {
  description = "AWS region for this lab"
  type        = string
  default     = "ap-south-1"
}
```

SAID IN CLASS

"Everything in Terraform is a block. So this is my block for variable, and then variable name."

Argument	What it does	Her verdict
`type`	What kind of value: `string` (letters), `number`, `bool`, `list(...)`, `map(...)`	*"Every time you need to mention variable type"*
`description`	A human note about the variable	Optional
`default`	The value used if nobody provides one	Optional — but best practice

SAID IN CLASS

"Sometimes you forget to mention this thing and you are creating your infrastructure by `terraform apply` … instead of failing your infrastructure, it will take the default value and it will execute it. So yeah, it's a best practice."

CLARIFIED

Clarified — technically, all three are optional. Terraform accepts a bare `variable "x" {}`. Leave out `type` and the variable accepts *any* type. Leave out `default` and the variable becomes **required** — `plan` / `apply` will **stop and ask you** for it (`var.x Enter a value:`), or fail in a CI pipeline where nobody can type. So her rule "always write `type` " is excellent practice (it catches mistakes early), and it's the answer to give in her course — just know it's a convention, not a parser rule.

Using it: anywhere a value goes, write `var.<name>` :

```
region = var.aws_region
bucket = var.bucket_name
```

The connection to Kubernetes/Helm she drew:

SAID IN CLASS

"Like in Kubernetes you will see `values.livenessProbe.something` — so you check the values file… same thing, you just need to remember how to use the variable."

Helm (Class 16)	Terraform
`values.yaml` holds the values	`variables.tf` declares, `terraform.tfvars` supplies
`{{ .Values.image.tag }}` in a template	`var.bucket_name` in a `.tf`

TRY IT YOURSELF

Where the value can come from (her repo `05-variables` uses option 2):

Way	Example
1. `default` in the block	`default = "ap-south-1"`
2. `terraform.tfvars` file (auto-loaded)	`aws_region = "us-east-1"`
3. Command-line flag	`terraform plan -var="environment=prod"`
4. A named vars file	`terraform apply -var-file="prod.tfvars"`

That's the dev/stage/prod trick from §3.1: **one** set of `.tf` files, **three** `.tfvars` files.

GOOD TO KNOW

About `.tfvars` and secrets. She said the `.tfvars` file is where *"files related to your secrets"* go. More precisely: `.tfvars` holds **variable values** — some harmless (region), some secret (a DB password). Because it *can* hold secrets, her `.gitignore` excludes `*.tfvars` and ships a safe `terraform.tfvars.example` instead.

11 🪣 Block 4 — `resource`: The Thing You're Creating

LOCK THIS IN

"*Resource means service — over the cloud platform.*" EC2 instance, S3 bucket, VPC, RDS, DynamoDB, IAM — every one is a **resource**.

```
resource "aws_s3_bucket" "provider_demo" {
  bucket_prefix = "session18-provider-"

  tags = {
    Name = "Session 18 Provider Demo"
  }
}
```

The two labels after `resource` are the part to really understand:

Part	Who decides it	Meaning
`resource`	Terraform	"This block creates something"
`"aws_s3_bucket"`	**Predefined** by the AWS provider	The resource **type**. "*You can't put your own keywords. You will see errors, errors and errors.*"
`"provider_demo"`	**You**	The resource's **local name** — how *your code* refers to it
`bucket_prefix`, `tags`	Predefined names, **your** values	The resource's arguments

CLARIFIED

Clarified — the local name is not the bucket's name. In class she said "*provider_demo — that I gave the name to my S3 bucket… this is the unique name.*" Two different names are in play:

```
resource "aws_s3_bucket" "provider_demo" {    ← Terraform-side name. Only exists in your code.
  bucket_prefix = "session18-provider-"        ← AWS-side name. What you see in the S3 console:
}                                                  session18-provider-20260929xxxxxxxx
```

`provider_demo` must be unique *within your Terraform code*. The **bucket** name must be unique *across every AWS account on Earth* (§17.4). Rename `provider_demo` and AWS never knows; rename the bucket and AWS creates a new one.

`bucket_prefix` — she explained it as "*the prefix we use for our bucket name, the same thing we mention while creating our S3 bucket manually.*" With a prefix, Terraform/AWS appends a unique random suffix, so the name is always available. (Keep that in mind for §17 — the demo used a fixed `bucket` name instead, and that's exactly what caused the pile-up.)

11.1 Tags = Kubernetes labels

> SAID IN CLASS
>
> *"Labels you learned inside Kubernetes. Where do we put the labels, guys?" — "In the pod." — "Same thing. When we call it a label there, here we call it a tag."*

Why tags matter, in her example: one `main.tf` creating **50 buckets** — 10 for dev, 10 for stage, 30 for prod. Tag each with its environment, and you can list "all the dev buckets" with one filter.

```
tags = {
  Name        = "anishka2307"
  Environment = "dev"
  ManagedBy   = "Terraform"
  Project     = "Session18"
}
```

Tags are **key = value** pairs, exactly like `labels:` in a pod's metadata.

---

12 📤 Block 5 — `output`: Did It Work? What Did It Make?

Once the bucket exists, how do you know? She listed **three** ways:

Way	How
1. The console	AWS console → S3 → the bucket list (before: only a *Create bucket* button; after: your bucket)
2. The AWS CLI	`aws s3 ls` — for when you don't have console (UI) access
3. **Terraform outputs**	Declare an `output` block; Terraform prints it after `apply`, and `terraform output` prints it any time

```
output "bucket_id" {
  value = aws_s3_bucket.provider_demo.id
}
```

She made the class trace the reference:

WATCH IT HAPPEN

How `aws_s3_bucket.provider_demo.id` is resolved.

```
Frame 1  aws_s3_bucket          → go to the resource blocks of this TYPE
Frame 2  .provider_demo         → pick the one with this LOCAL NAME
Frame 3  .id                    → read its "id" attribute (for S3: the bucket's name)
Frame 4  value printed after apply:   bucket_id = "session18-provider-xxxxxxxx"
```

The pattern is always `<TYPE>.<LOCAL_NAME>.<ATTRIBUTE>` — "*resource dot resource name*", then the field you want.

A student's worry: *we never wrote an `id` in the resource, so won't this error?* Her answer:

SAID IN CLASS

"ID is always there… If S3 is created there are almost 20 parameters. You can't mention 20 parameters over here. You just need to mention what you need. Other than that, all the parameters are already there. You can use it directly."

WHY IT WORKS

Arguments vs attributes. Arguments are what you *send* (`bucket_prefix` , `tags`). **Attributes** are what AWS *sends back* once the thing exists (`id` , `arn` , `region` , `bucket_domain_name` …). That's why the plan in your screenshot says `bucket_arn = (known after apply)` — the ARN doesn't exist until AWS creates the bucket.

CLARIFIED

Small clarification. She said the output's name `bucket_id` "*is also a predefined keyword.*" The **attribute** (`.id`) is predefined; the **output's name** (`"bucket_id"`) is yours — your own `outputs.tf` names them `bucket_name` , `bucket_arn` , `bucket_region` , and they work fine.

---

13 🧩 The Whole File, Together

This is the exact file she walked through (`03-providers/main.tf` in her repo — and copied into your `03-providers/main.tf`):

```hcl
# ── 1. Which Terraform, which providers ──────────────────
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

# ── 2. Configure the AWS provider ────────────────────────
provider "aws" {
  region = var.aws_region            # ← uses block 3
}

# ── 3. An input ──────────────────────────────────────────
variable "aws_region" {
  description = "AWS region for this lab"
  type        = string
  default     = "ap-south-1"         # Mumbai
}

# ── 4. The actual infrastructure ─────────────────────────
resource "aws_s3_bucket" "provider_demo" {
  bucket_prefix = "session18-provider-"
  tags = {
    Name = "Session 18 Provider Demo"
  }
}

# ── 5. What to print back ────────────────────────────────
output "bucket_id" {
  value = aws_s3_bucket.provider_demo.id   # ← reads block 4
}
```

Her summary, in order:

> **SAID IN CLASS**
>
> *"First you need to mention your Terraform version. Second, you need to mention your provider — AWS. Then you need to install your CLI for the provider. And after that, the actual code will start from here itself… resource type and then resource name… and then output block. Done."*

> **LOCK THIS IN**
>
> **The three things to really know:** "*Resource, variable, output.*" The `terraform {}` and `provider` blocks are near-identical boilerplate you copy each time.

14 ⌨ The Commands — `init` to `destroy`

SAID IN CLASS

"Rarely seven commands are there, but only four to five commands are the main commands; the others we use as a flag."

#	Command	What it does	Touches AWS?
1	`terraform init`	Initialise the folder: download the provider into `.terraform/`, write the lock file	No (talks to the Registry)
2	`terraform fmt`	Rewrite `.tf` files into the standard format/indentation	No
3	`terraform validate`	Check the code: real keywords, valid syntax, valid references	No
4	`terraform plan`	Preview: what will be added / changed / destroyed	Reads
5	`terraform apply`	Do it — create/update the real infrastructure (asks `yes`)	**Writes**
6	`terraform output`	Print the `output` values	No (reads state)
7	`terraform state list`	List every resource Terraform manages	No (reads state)
8	`terraform state show <type>.<name>`	Show every attribute of one resource	No (reads state)
9	`terraform plan -destroy`	Preview what `destroy` would delete	Reads
10	`terraform destroy`	Delete everything this configuration manages (asks `yes`)	**Deletes**

14.1 `init` — like `git init`

SAID IN CLASS

"After running the command `git init`, if you do the `ls -la`, which folder is created?" — `.git`. "So your first command for Terraform… `terraform init`. After running this command there is a folder that will be created — `.terraform`."

Run it once per folder, and again whenever you add a provider. It's safe to repeat.

14.2 `fmt` — the auto-tidy

SAID IN CLASS

"Once I execute my `terraform fmt` command, it will automatically make sure that the indentation is correct and the format is correct for my file… If it is not in a good shape, it will make it in a good shape."

She contrasted it with Kubernetes, where a YAML indentation slip means "*YAML is not parsing*" and the pod never starts. Terraform will just fix the layout for you.

CLARIFIED

Clarified — `fmt` fixes *layout*, not *mistakes*. She also said that if you typed two curly braces, "*Terraform fmt will give you error… it will automatically fix it.*" Only half true: `fmt` **re-indents, aligns the `=` signs and normalises spacing**, and prints the names of files it changed. If the syntax is actually broken (an extra `}`), `fmt` **reports an error and changes nothing** — you fix it by hand. Spotting wrong *keywords* is `validate`'s job, not `fmt`'s.

```
# before fmt  (her providers.tf, all on one line)
provider "aws" { region = var.aws_region }

# after fmt  (your providers.tf)
provider "aws" {
  region = var.aws_region
}
```

14.3 `validate` — the spell-checker

SAID IN CLASS

"It will validate your file — all the predefined keywords that you added inside your Terraform file actually exist or not."

On success: `Success! The configuration is valid.` On failure it points at the **file, line and column**, and often suggests the fix — *"you just have to copy and paste it. It's so easy."* (Live example in §17.3.)

WHY IT WORKS

What `validate` can't catch. It never talks to AWS. So it can't know that a bucket name is taken, or that your keys lack permission, or that your keys are wrong. All three of those hit the class in §17 — **after** a clean `validate`. That's why the full sequence matters.

14.4 `plan` and `apply`

SAID IN CLASS

"*`terraform plan` will give you the output — what output actually you will get after the `terraform apply` command… We are going to add two resources, update and delete four resources… And if you are satisfied with that plan, you can do `terraform apply`.*"

Reading a plan (the colours and symbols she pointed out):

Symbol	Colour	Meaning
`+`	green	create
`-`	red	destroy
`~`	yellow	update in place
`-/+`	—	destroy and re-create (replace)

And the last line is the summary: `Plan: 1 to add, 0 to change, 0 to destroy.`

`apply` shows the plan again and stops at:

```
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```

Anything other than the full word `yes` cancels. "*Before yes, it will not do it.*"

WATCH IT HAPPEN

What happens between `terraform apply` and "Apply complete!"

```
Frame 1  Read every *.tf file in the folder → one configuration
Frame 2  Read terraform.tfstate → "what do I already manage?" (empty on first run)
Frame 3  Provider logs in to AWS with your configured keys
Frame 4  Compare: wanted (config) vs have (state + real AWS) → the plan
Frame 5  Show the plan, wait for "yes"
Frame 6  Call the AWS API:  CreateBucket "anishka2307"   → aws_s3_bucket.anishka2307: Creating...
Frame 7  AWS replies with id, arn, region…               → Creation complete after 2s
Frame 8  Write all of it into terraform.tfstate
Frame 9  Print outputs (bucket_name, bucket_arn, bucket_region)
```

GOOD TO KNOW

From her repo (`07-init-plan-apply`) — the CI/CD version. `terraform plan -out=tfplan` saves the reviewed plan; `terraform apply tfplan` then applies *exactly that plan*, not a freshly computed one. That's how pipelines make sure what was reviewed is what runs.

14.5 `output` , `state list` , `state show`

```
terraform output                    # all outputs
terraform output bucket_name        # just one
terraform state list                # → aws_s3_bucket.anishka2307
terraform state show aws_s3_bucket.anishka2307   # every attribute of that one resource
```

SAID IN CLASS

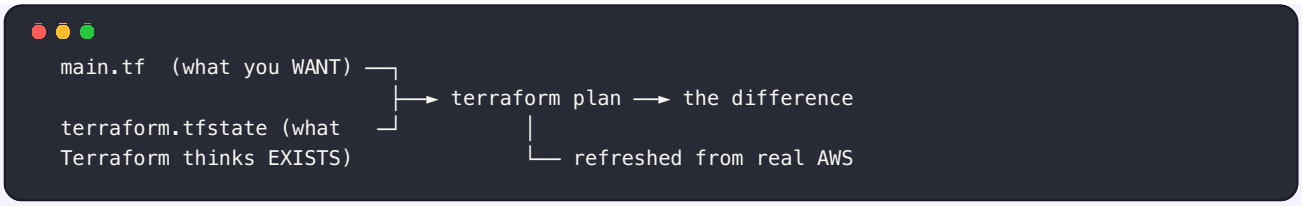
"Suppose I created four resources through Terraform… if you want to see actually what we created inside the S3 bucket, there is a second command, slightly changed: `terraform state show` … resource dot resource name."

14.6 The state file — the thing behind `state list`

She used the `state` commands but didn't stop to define state; her repo's `09-state` folder does.

PICTURE THIS

Terraform's diary. `terraform.tfstate` is a JSON file where Terraform writes down *everything it built*: " `aws_s3_bucket.anishka2307` is the real bucket `anishka2307` , in `ap-south-1` , with this ARN." Next time you run `plan` , it compares **your code** with **the diary** (and refreshes the diary from AWS) to work out the difference.



Lose the diary and Terraform forgets it owns the bucket — it would try to create it again.

WATCH OUT

Two state rules (from her `09-state/README.md`): never commit `terraform.tfstate` / `terraform.tfstate.*` to Git (it can contain account details and secrets), and **never hand-edit it** — use `terraform state …` commands. Your `.gitignore` already covers both.

TRY IT YOURSELF

Your own state files tell the story of your class. Your `terraform.tfstate.backup` (written at 13:16) still holds the bucket `anishka2307` in `ap-south-1` with tags `Environment=dev, ManagedBy=Terraform, Project=Session18`; your current `terraform.tfstate` is **empty** (serial 5, no resources). That's the before/after of `terraform destroy` — Terraform keeps the previous version as `.backup` each time it writes.

---

15 ☠️ `terraform destroy` — The Dangerous One

```
terraform plan -destroy    # preview: what WOULD be deleted
terraform destroy          # actually delete — asks for 'yes'
```

SAID IN CLASS

"If you will run this command, whatever you have deployed will delete everything… all the resources will delete. Terraform is actually a very advanced concept — you put one command here and you can't take it back."

That's why:

SAID IN CLASS

"If you become a DevOps engineer, no one will give you access directly."

And it's why today's IAM user was locked to S3 only — *"I know you will try some new commands over there, and maybe you can access the whole AWS."*

LOCK THIS IN

The habit: always `terraform plan -destroy` (read it!) before `terraform destroy`, and always `terraform plan` before `apply`. Her repo's `08-destroy` practice question: "*Why should we run `plan -destroy` before deleting production infrastructure?*" — because `destroy` deletes **everything in that state**, not just the thing you're thinking of. Read the list first.

GOOD TO KNOW

Why your destroy worked even on a non-empty bucket: your `main.tf` has `force_destroy = true`. Without it, AWS refuses to delete a bucket that still contains objects, and `destroy` fails with *BucketNotEmpty*. Handy for a demo; think twice before using it on a real data bucket.

---

16 🔑 IAM Live — Making the Student User and Configuring the CLI

The one or two students who had their own AWS accounts followed along. Everyone else watched.

WATCH IT HAPPEN

Creating a limited user for the batch.

```
Frame 1  AWS console → search "IAM" → Users → Create user
Frame 2  Username:   devops-students-batch-a1   (something like that)
Frame 3  Permissions → no group yet → Create group (give it a name)
Frame 4  Attach a policy to the group: search "S3" → AmazonS3FullAccess
Frame 5  Select the group → Next → Create user
Frame 6  Open the user → Security credentials → Create access key
Frame 7  You get TWO strings: an Access Key ID and a Secret Access Key
         (the secret is shown ONCE — save it now or make a new key)
```

SAID IN CLASS

"In that full access, you can list, you can create, you can update and you can delete — only S3 buckets."

CLARIFIED

A small contradiction worth noticing. Earlier she said *"I will give you the access to only create the S3 bucket, not even deleting."* But `AmazonS3FullAccess` **includes** delete — which is exactly why everyone's `terraform destroy` worked at the end. A true "create-only" user would need a custom policy allowing `s3:CreateBucket` but not `s3:DeleteBucket` (and then `destroy` would fail with Access Denied).

She hid the secret on screen while saving it, then shared both keys on the class chat / notice board — and pre-empted the questions:

SAID IN CLASS

(Hindi, translated) "*This is the access key. And the second is the secret access key — let me label them now, otherwise someone will ask me 'what's this, what's that'.*"

16.1 `aws configure`

```
aws configure
AWS Access Key ID [None]:      <paste the access key>
AWS Secret Access Key [None]:  <paste the secret key>
Default region name [None]:    us-east-1
Default output format [None]:  json
```

SAID IN CLASS

(Hindi, translated) "*Copy the whole access key from here, paste it here, press Enter. Then it asks for the secret key — paste the second one I sent… region: us-east-1… output format: type json.*"

CLASS MOMENT

One student asked whether they needed to set up an **MCP server** for this. No — "*you don't need any MCP kind of a server.*"

PICTURE THIS

Two keys, two jobs.

Key	Like	Safe to show?
Access Key ID (`AKIA…`)	Your username	Not secret by itself, but don't post it
Secret Access Key	Your password	**Never** — anyone with both *is* you, with your permissions

`aws configure` stores them in `~/.aws/credentials` and the region/format in `~/.aws/config`. Both the AWS CLI and Terraform's AWS provider read from there.

WATCH OUT

Never paste these into a `.tf` file, a README, a screenshot or a commit. Bots scan public GitHub for `AKIA…` strings within minutes. (Your screenshots and files are clean — checked.)

17 🚀 The Live Demo — `terraform-s3-demo`

SAID IN CLASS

"*Go to this terraform-s3-demo folder… git clone and go to this folder.*" Then, after a moment of confusion about which session folder: "*Which session? Even I don't know — it's 18, right? Go into 18.*"

17.1 The files

Her demo folder (and your copy) split the blocks into standard files (§7.2). Your version:

```hcl
# terraform.tf
terraform {
  required_version = ">= 1.6.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

# providers.tf
provider "aws" {
  region = var.aws_region
}

# variables.tf
variable "aws_region" {
  type        = string
  description = "AWS region where the S3 bucket will be created."
  default     = "ap-south-1"
}
variable "bucket_name" {
  type        = string
  description = "Name of the S3 bucket."
  default     = "anishka2307"          # hers ended as "devops553"
}

# main.tf
resource "aws_s3_bucket" "anishka2307" {
  bucket        = var.bucket_name
  force_destroy = true
  tags = {
    Name        = var.bucket_name
    Environment = "dev"
    ManagedBy   = "Terraform"
    Project     = "Session18"
  }
}

# outputs.tf
output "bucket_name" {
  type        = string
  description = "Name of the S3 bucket."
  value       = aws_s3_bucket.anishka2307.bucket
}
output "bucket_arn" {
  type        = string
  description = "ARN of the S3 bucket."
  value       = aws_s3_bucket.anishka2307.arn
}
output "bucket_region" {
  type        = string
  description = "AWS region of the S3 bucket."
  value       = aws_s3_bucket.anishka2307.region
}
```

PICTURE THIS

ARN, since it's in the outputs. An **ARN** (Amazon Resource Name) is AWS's globally unique ID for anything: `arn:aws:s3:::anishka2307`. Format: `arn:partition:service:region:account:resource` — S3 bucket ARNs leave region and account blank (the two empty `::`), because bucket names are already global.

17.2 `init`, then `fmt`

```
terraform init      # "Initializing the backend… Terraform has been successfully initialized!"
ls -la              # → .terraform/ and .terraform.lock.hcl now exist
terraform fmt       # → providers.tf
```

SAID IN CLASS

(Hindi, translated) "*Did `init`? 'Initializing the backend…' — green, green, something like that came? If you didn't ask now, don't ask later… Do `ls -la` — you should see `.terraform`.*"

`fmt` printed **`providers.tf`** — because in her repo that file was deliberately written on one line (`provider "aws" { region = var.aws_region }`), and `fmt` expanded it. Your `providers.tf` is the expanded, formatted version.

17.3 Break it on purpose → `validate`

She had everyone add a typo — `aws_s3_bucket` → `aws_s3_bucket1` — then ran `terraform validate`. What she read out from her screen:

```
$ terraform validate

╷
│ Error: Invalid resource type
│
│   on … line 4 …
│
│ The provider hashicorp/aws does not support resource type "aws_s3_bucket1".
│ Did you mean "aws_s3_bucket"?
╵
```

SAID IN CLASS

"*Did you mean `aws_s3_bucket`? — that is your actual fix for the file.*" Remove the `1`, run again:

```
$ terraform validate
Success! The configuration is valid.
```

NOTE

Where exactly was the typo? She said "*go to this main file*", but the error she read pointed at **`outputs.tf`, line 4** — which in the demo is the `value = aws_s3_bucket.devops553.bucket` line. Either way the lesson is the same: `validate` names the **file, line and the bad keyword**, and suggests the right one. (If you typo the *reference* in `outputs.tf` rather than the resource block, you may instead see "*Reference to undeclared resource*" — Lab 2 step 7 lets you see both.)

17.4 `plan`, `apply` — and three real errors

```
$ terraform plan
  + resource "aws_s3_bucket" "…" { … }
Plan: 1 to add, 0 to change, 0 to destroy.
```

SAID IN CLASS

(Hindi, translated) "*In the plan we're adding only one resource, destroying nothing, updating nothing. You'll see a plus symbol — `+ create`. When you delete, everything comes in red with a minus.*"

Then `terraform apply` → `yes` — and the room hit, one after another, the three classic first-day errors:

#	Error (as heard)	Real cause	Fix
1	"*Error… retrieving AWS account details*" (at **plan**)	The provider couldn't log in — `aws configure` not done, or keys pasted wrong	Re-run `aws configure` carefully; test with `aws sts get-caller-identity`
2	"*Access Denied*" on `CreateBucket`	The keys belonged to the **wrong IAM user** — she checked IAM live and realised "*then we were using the wrong keys, guys!*"	She re-issued the right user's keys (the one with `AmazonS3FullAccess`)
3	"*The requested bucket name is not available*"	**S3 bucket names are global.** Every student was applying the *same* name, `yatri1107` — the first one to succeed took it for the whole planet	Change `bucket_name` to something unique

CLASS MOMENT

"I'm so famous." Even her own attempts kept colliding. `yatri1107` → taken. `yatri1107-2000` → taken. `nancy1107` → "*What a joke. Someone has taken this name. I'm so famous.*" `devops-mc…` → taken. Finally `devops553` went through — "*It's so random that it is not available anywhere.*" Her advice for everyone else: **your name + your birth date**. Which is exactly what you did: `anishka2307`.

WHY IT WORKS

Why the name clash happened, and how her other folders avoid it. S3 bucket names live in **one global namespace** shared by every AWS account (3–63 chars, lowercase letters, digits, hyphens, dots). The demo used a fixed `bucket = var.bucket_name`, so 50 people applying the same default all fought over one name. Her folders `01`–`09` use `bucket_prefix = "session18-…-"` instead, which lets AWS/Terraform append a unique random suffix — no clashes, ever. Use a prefix whenever you don't need an exact name.

CLARIFIED

The region mix-up. After apply, she opened the S3 console in **us-east-1 (N. Virginia)** and couldn't see anyone's buckets — because everyone had typed `us-east-1` in `aws configure`, but the Terraform code says `region = var.aws_region` with default `ap-south-1`. **The provider's `region` argument wins** over the CLI profile's default region. So every bucket was created in **Mumbai**. She worked it out: "*no problem, because we have a file where we have a default region — so it has overwritten it.*" (Your outputs confirm it: `bucket_region = "ap-south-1"`.)

17.5 Your screenshot — the apply

```
(.venv) (base) anishkanase@Anishkas-MacBook-Air terraform-s3-demo % terraform apply
Changes to Outputs:
  + bucket_arn    = (known after apply)
  ~ bucket_name   = "yatri1107" -> "anishka2307"

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

aws_s3_bucket.anishka2307: Creating...
aws_s3_bucket.anishka2307: Creation complete after 2s [id=anishka2307]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
```

LOCK THIS IN

What this proves. Your `terraform apply` created the bucket: `aws_s3_bucket.anishka2307: Creation complete after 2s [id=anishka2307]` and `Apply complete! Resources: 1 added, 0 changed, 0 destroyed.` Look at the outputs diff too: `bucket_arn` is `+ (known after apply)` (the attribute didn't exist yet — §12), and `bucket_name` is `~ "yatri1107" -> "anishka2307"`. That `~` means your state already had an output called `yatri1107` from an **earlier apply that failed** on the taken name — you then switched to your own name and re-applied. The whole §17.4 saga, in one line.

17.6 Check, then clean up

```
terraform output                # bucket_arn / bucket_name / bucket_region
aws s3 ls                       # lists buckets from the CLI
terraform state list            # aws_s3_bucket.anishka2307
terraform plan -destroy         # preview
terraform destroy               # yes
```

CLASS MOMENT

The bucket census. "*How many people created?*" She refreshed S3: about **25 buckets** — from a class of ~50. (Hindi, translated:) "*25 people out of 50 didn't do it — and one of those is me. So think: even if we give credentials, what happens?*" Then the bucket-name roll-call: a "*spirit*" bucket, a "*Tanya*" bucket ("*who is Tanya?*"), a "*Yatri's fan*" bucket, "*Adish*"… and one last "*1107*" bucket that someone made at random.

CLASS MOMENT

"Don't do it, I'll do it." When people started destroying, she switched to deleting the leftovers manually from the console herself — "*guys, don't do it, I will do it manually.*"

17.7 Your screenshot — the destroy

```
(.venv) (base) anishkanase@Anishkas-MacBook-Air terraform-s3-demo % terraform destroy

Changes to Outputs:
  - bucket_arn    = "arn:aws:s3:::anishka2307" -> null
  - bucket_name   = "anishka2307" -> null
  - bucket_region = "ap-south-1" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_s3_bucket.anishka2307: Destroying... [id=anishka2307]
aws_s3_bucket.anishka2307: Destruction complete after 1s

Destroy complete! Resources: 1 destroyed.
(.venv) (base) anishkanase@Anishkas-MacBook-Air terraform-s3-demo %
```

LOCK THIS IN

What this proves. You cleaned up properly: the destroy plan shows all three outputs going `-> null`, AWS's warning "*There is no undo. Only 'yes' will be accepted to confirm.*", then `aws_s3_bucket.anishka2307: Destruction complete after 1s` and `Destroy complete! Resources: 1 destroyed.` Your empty `terraform.tfstate` agrees. Nothing left running on her account.

---

18 🧐 Reviewing Your Own Files

`terraform-s3-demo/` **— good work.**

- ✅ **Unique, readable bucket name** (`anishka2307`) — following her name+birthdate advice, and you used it for *both* the resource's local name and the bucket name, which keeps `state list` easy to read.
- ✅ **Bucket name in a variable**, not hard-coded in `main.tf` — the right instinct for multi-environment code.
- ✅ **Tags** with `Environment`, `ManagedBy`, `Project` — `ManagedBy = "Terraform"` is a real-world convention that tells colleagues "*don't edit this by hand in the console.*"
- ✅ `.gitignore` excludes `.terraform/`, `*.tfstate`, `*.tfstate.*`, `*.tfvars` and plan files — so your state (which contains your account details) can't be committed by accident. Keep it that way.
- ✅ **You destroyed what you created** — both screenshots and the state file confirm it.

Small things to know:

GOOD TO KNOW

`type` **inside** `output` **blocks.** Your `outputs.tf` (copied from her demo) puts `type = string` in each output. It ran fine on your Terraform 1.16.4. Classic Terraform outputs only take `value`, `description`, `sensitive` and `depends_on` — `type` is a `variable` argument — so if you ever run this on an older Terraform and `validate` complains about *Unsupported argument* "*type*", just delete those lines; the outputs work without them.

GOOD TO KNOW

The README mentions a file that isn't there. The `README.md` in `terraform-s3-demo/` (identical to hers) lists `terraform.tfvars` in the project structure, but the folder has none — the values come from the `default`s in `variables.tf`. Not a bug; just don't go looking for it.

`03-providers/main.tf` — two fixes.

WATCH OUT

Stray shell commands at the end of the file. After the `output` block, your file ends with:

```
ls -la
terraform fmt
terraform validate
```

Those are terminal commands, not HCL. Terraform reads *every* line of a `.tf` file, so `terraform validate` in this folder would fail with "*Argument or block definition required*". Move notes like this into a comment (`# ls -la`) or into your README.

WATCH OUT

The commented-out Azure provider has the wrong name. You wrote (commented):

```
// azure = {
//  source  = "hashicorp/azure"
//  version = "~> 6.0"
//}
```

The Azure provider is `hashicorp/azurerm` (local name usually `azurerm`), and it has its **own** version numbers — don't copy AWS's `~> 6.0`. If you uncomment it as-is, `terraform init` will fail to find the provider. (Good that you commented it out — and great that you tried the multi-cloud idea from §5.)

`Readme.MD` — just the two screenshots, which is exactly what she asked for (*screenshot every cloud step, save it in the folder*). Consider adding one line under each saying what it shows.

---

19 🏋 Practice Labs

You no longer have her credentials. Two options: create your own **AWS Free Tier** account (a few empty S3 buckets cost effectively nothing — but **always destroy**), or do the *no-cloud* parts below, which need no keys at all.

DO THIS YOURSELF

Lab 2 — Write the five blocks from memory (no AWS needed).

1. `mkdir tf-lab && cd tf-lab`

2. Create `main.tf` with: a `terraform {}` block (`>= 1.6.0` , `hashicorp/aws ~> 6.0`), a `provider "aws"` using `var.aws_region` , a `variable "aws_region"` (type string, default `ap-south-1`), a `resource "aws_s3_bucket" "lab"` with a `bucket_prefix` and two tags, and an `output "bucket_id"` .
3. `terraform init` → check `ls -la` shows `.terraform/` and `.terraform.lock.hcl` .
4. Mess up the indentation on purpose, then `terraform fmt` — note which file names it prints.
5. `terraform validate` → expect `Success! The configuration is valid.`
6. Change `aws_s3_bucket` to `aws_s3_bucket1` → `validate` again → read the "*Did you mean*" line → fix it.
7. Change `output` to reference `aws_s3_bucket.wrongname.id` → `validate` → read the "*Reference to undeclared resource*" error.

All of this runs without credentials — `init` , `fmt` and `validate` never log in to AWS.

DO THIS YOURSELF

Lab 3 — The full lifecycle (needs your own AWS keys).

1. `aws configure` → `aws sts get-caller-identity` (must succeed first).
2. In your Lab 2 folder: `terraform plan` → find the `+` line and `Plan: 1 to add, 0 to change, 0 to destroy.`
3. `terraform apply` → type `yes` . Note the generated bucket name in the output.
4. `terraform output` · `terraform output bucket_id` · `aws s3 ls` .
5. `terraform state list` → `terraform state show aws_s3_bucket.lab` . Find `arn` , `region` , `tags` .
6. Change a tag's value in `main.tf` → `terraform plan` → you should see `~ update in-place` and `0 to add, 1 to change` . Apply it.
7. `terraform plan -destroy` → read it → `terraform destroy` → `yes` .
8. `terraform state list` → should print nothing.

DO THIS YOURSELF

Lab 4 — Reproduce the name clash, then fix it with a prefix.

1. In a copy of `terraform-s3-demo` , set `default = "test"` for `bucket_name` (a name certainly taken by someone in the world).
2. `terraform apply` → read the "*requested bucket name is not available*" error. Note that `validate` and `plan` both passed.
3. In `main.tf` , replace `bucket = var.bucket_name` with `bucket_prefix = "anishka-lab-"` . (Update the `Name` tag too.)
4. `terraform apply` → it succeeds with a random suffix. `terraform destroy` .

DO THIS YOURSELF

Lab 5 — One code, three environments (her `05-variables` folder).

1. Copy `05-variables` from her repo. `cp terraform.tfvars.example terraform.tfvars` .
2. `terraform init && terraform plan` → note the prefix `student-project-dev-…` and tag `Environment = "dev"` .

3. Edit `terraform.tfvars` : `environment = "test"` → `terraform plan` again. What changed in the plan? (Changing the prefix forces a **replace**, `-/+` .)
4. Make `dev.tfvars` and `prod.tfvars` and try `terraform plan -var-file=prod.tfvars` .
5. Try `terraform plan -var="environment=stage"` — the command line wins over the file.

20 🔭 Next Class Preview — A Whole VPC

SAID IN CLASS

"We just created one resource, S3 bucket. Next class we are going to create five resources, and they will connect with each other. We are going to create a VPC."

Resource	One-line meaning
VPC	Your own private network inside AWS
Internet gateway	The VPC's door to the internet
Subnets — public and private	Smaller rooms inside the network; public ones can reach the internet, private ones can't directly
Security groups	Firewalls around your servers
Route table + **route table association**	The signposts that say "traffic for the internet → go via the gateway", attached to specific subnets

SAID IN CLASS

"*Hope you enjoyed today's session.*" She stayed back 15 minutes for doubts, then revoked the credentials.

21 🎯 Quiz & Q&A Recap

Question	Asked to	Answer given	Verdict
What do you need first to create an EC2 instance?	Class	Log in to the AWS account (username, password, maybe MFA)	✅
Which port for SSH?	Class	22 (some said "20")	✅ 22
Which port for HTTPS?	Class	443	✅
Why not use Helm instead of Terraform?	Student's question	Helm/K8s *deploy and orchestrate* the app; Terraform *provisions* all the infrastructure (EKS included)	✅
Is Kubernetes a sub-part of Terraform?	Student's question	"*No — both are completely different things.*"	✅
What is a resource?	Class	A service on the cloud — EC2, S3, VPC, RDS, DynamoDB, IAM	✅
What is Azure's storage service called?	Class	No audible answer	❓ (Azure Blob Storage / Storage Account — §7.3)
Where do we put labels in Kubernetes?	Class	"*In the pod.*"	✅
What is `provider_demo` in `aws_s3_bucket.provider_demo.id`?	Class	The resource name	✅ (the **local** name — §11)
What does `git init` create (the analogy for `terraform init`)?	Class	`.git` → so `terraform init` creates `.terraform`	✅
What does `terraform plan -destroy` output?	"*Anyone can guess?*"	The list of resources `destroy` would delete	✅
How many have AWS access / tried Terraform before?	Show of hands	One or two; one had run a single file	—
Do we need an MCP server for `aws configure`?	Student's question	No	✅

INTERVIEW ALERT

Interview alerts from this class.

- *What's the difference between Terraform and Kubernetes?* → Provisioner vs orchestrator (§4).
- *What's* `terraform plan` *vs* `apply`*?* → Preview vs execute (§14.4).
- *What is Terraform state and why not commit it?* → Terraform's record of real infra; can hold secrets (§14.6).
- *What does* `required_version = ">= 1.6.0"` *do?* → Enforces a minimum CLI version — fails if lower (§8.1).
- *Why multi-cloud?* → No single point of failure (§5).

22 📝 Homework

HOMEWORK

1. Five READMEs on five AWS services — the prerequisite for next class.

One README each, *what you learned about*: **EC2** (compute) · **S3** (storage) · **VPC** (networking) · **IAM** (access) · **DynamoDB and RDS** (databases).

"Next session, for 10 minutes I will ask the questions, and if you're not able to fulfil this prerequisite then you need to go outside." S3 and IAM were covered today, so really *"you have to see only three services."*

HOMEWORK

2. Today's Terraform assignment — done in class while the keys were live, **with screenshots** of every cloud step saved in the folder (✓ done: `image.png` , `image-1.png`).

HOMEWORK

3. Check the updated task sheet (linked from the DevOps Heroes root `README.md`) for all 21 sessions, and watch for the **final project** README.

LOCK THIS IN

A starter answer for the five services (try writing yours first):

Service	What it is	One-line analogy	Terraform resource
EC2	Virtual servers you rent by the hour; pick an AMI, an instance type, a key pair and security groups	Renting a flat instead of buying a house	`aws_instance`
S3	Object storage — files ("objects") in **buckets**; bucket names are globally unique	A bottomless storage locker with a global address	`aws_s3_bucket`
VPC	Your private, isolated network in an AWS region, split into subnets	Your own gated colony inside the city	`aws_vpc`
IAM	Who can do what: users, groups, roles, and **policies** (JSON permission lists)	The security desk that hands out key cards	`aws_iam_user` , `aws_iam_policy`
RDS	Managed **relational** databases (Postgres, MySQL…) — AWS handles backups and patching	A librarian who keeps your tables in order	`aws_db_instance`
DynamoDB	Managed **NoSQL** key-value/document database, scales automatically	A huge, fast pigeon-hole cabinet	`aws_dynamodb_table`

LOCK THIS IN

Her repo's practice questions, answered.

- *What happens if 10 engineers manually create the same infrastructure?* → 10 slightly different versions — different names, settings, forgotten tags — and nobody knows which is "right". IaC gives one reviewed definition.
- *How can Git help with infrastructure?* → History of every change, pull-request review before anything hits the cloud, and rollback by reverting a commit.
- *What if we need the same infrastructure in another environment?* → Same `.tf` files, a different `.tfvars` (§10).
- *Why does changing a variable change the desired infrastructure?* → Variables feed resource arguments; a new value means a new desired state, so `plan` shows a diff.
- `plan` *vs* `apply` *?* → `plan` previews and changes nothing; `apply` executes (after `yes`).
- *What does* `destroy` *do?* → Deletes every resource in that configuration's state.

23 Cheat-Sheet

Setup

Command	What it does
`terraform version`	Show the installed Terraform CLI version
`aws --version`	Check the AWS CLI is installed
`aws configure`	Save access key, secret key, default region, output format into `~/.aws/`
`aws sts get-caller-identity`	"Who am I logged in as?" — test credentials
`aws s3 ls`	List your S3 buckets

Terraform workflow

Command	What it does
`terraform init`	Download providers → `.terraform/`, write `.terraform.lock.hcl`
`terraform fmt`	Auto-format `.tf` files (prints changed files)
`terraform validate`	Check syntax, keywords, references — no AWS call
`terraform plan`	Preview `+` create / `~` update / `-` destroy
`terraform plan -out=tfplan` → `terraform apply tfplan`	Save a reviewed plan and apply exactly that (CI/CD)
`terraform apply`	Create/update for real — type `yes`
`terraform output` / `terraform output <name>`	Print output values
`terraform state list`	List managed resources
`terraform state show <type>.<name>`	All attributes of one resource
`terraform show`	Show the whole state in readable form
`terraform plan -destroy`	Preview what `destroy` would delete
`terraform destroy`	Delete everything in this state — type `yes`

The five blocks

Block	Shape	Key point
`terraform`	`required_version`, `required_providers { aws = { source, version } }`	Minimum CLI version; which provider plugins
`provider`	`provider "aws" { region = … }`	Which cloud, which region. No keys in here!
`variable`	`variable "name" { type, description, default }`	Use as `var.name`. `type` always (her rule)
`resource`	`resource "<TYPE>" "<LOCAL_NAME>" { args }`	Type is predefined; local name is yours
`output`	`output "name" { value = <TYPE>.<LOCAL_NAME>.<ATTR> }`	Attributes like `id`, `arn` exist automatically

Files

File	Commit to Git?
`*.tf`	✅ yes
`.terraform.lock.hcl`	✅ yes (pins provider versions for the team)
`terraform.tfvars.example`	✅ yes
`terraform.tfvars` / `*.tfvars`	❌ no — may hold secrets
`terraform.tfstate`, `*.tfstate.*`	❌ never
`.terraform/`	❌ no — re-created by `init`

Concepts

Term	Meaning
IaC	Infrastructure as Code — define infra in files, not clicks
Provisioning	Creating infrastructure in the cloud
Provisioner vs orchestrator	Terraform builds the infra; Kubernetes runs containers on it
Provider	Plugin that lets Terraform talk to one platform's API (AWS, Azure, GCP…)
Resource	One cloud service/object Terraform manages
State	`terraform.tfstate` — Terraform's record of what it built
`>=` / `~>`	Minimum version / "this major version only"
HCL	HashiCorp Configuration Language — the `.tf` syntax
Tags	Key = value labels on cloud resources (like K8s labels)
ARN	Amazon Resource Name — global ID of an AWS resource
Multi-cloud	Using several clouds to avoid a single point of failure
IAM user / group / policy	Identity / a set of identities / the list of allowed actions
Access key + secret key	Programmatic "username + password" for AWS
`bucket` **vs** `bucket_prefix`	Exact global name (can clash) vs prefix + random suffix (never clashes)
`force_destroy`	Let `destroy` delete a bucket even if it has objects

24 🗒️ Notes on the Transcript Itself

The instructor. The transcript header credits **"Ritesh prajapati"** — that's the developer of the *Scaler++ Chrome extension* that downloaded the file, **not** the instructor. This class was taught by **Nensi Ravaliya** (heard as "*Nancy*"; she uses `Nancy1107` as a bucket name and `yatri-nancy` as an instance name).

The title is accurate. *Terraform & Infrastructure as Code* is what was taught — unlike Classes 12–14, no mismatch here. Her repo calls it `session18-terraform-iac`.

Transcription quality. `Model use: groq, whisper-large-v3-turbo`. The English concept part (§2–§15) is clean. The hands-on half is much worse:

- **The IAM / credential-sharing stretch and most of the demo** were spoken in Hindi and transcribed in **Devanagari script**, often phonetically garbled. The meaning was recoverable from context and is translated inline (§16, §17.2, §17.4, §17.6). Some short fragments — "*A B C T, don't add his name*", "*you will give an AWS free voucher 50%*", "*these students are looking for who know themselves*" — have no recoverable meaning and are left out.
- **A short stretch near the start** ("*Everyone, everyone… If you don't know, then the telephone will not be able to get up… Corporate.*") is crosstalk while she was dealing with the room; skipped.
- **The bucket-name collision section** is a repetitive loop of "*not available*", partial names and laughter; the sequence of names in §17.4 is reconstructed from it, and the final `devops553` is confirmed by her repo's `main.tf` (`resource "aws_s3_bucket" "devops553"`).
- Ending "*Thank you.*" — skipped.

Cross-referenced against files, not audio: every code block in §8–§13 and §17.1 comes from her repo or your folder; the bucket name, region, tags and destroy are confirmed by your `terraform.tfstate.backup`, `terraform.tfstate` and screenshots; the `validate` error location she read out (outputs.tf line 4) matches the `value = …` line of the demo's `outputs.tf`. **No access keys, secret keys or account IDs are reproduced** — the state backup contains account-identifying data and was deliberately not quoted.

Garbled-term corrections:

Transcript says	Actually
"terraform-iec"	`terraform-iac`
"telephone" / *"telephone validate"* / *"TAFO"*	**Terraform** / `terraform validate`
"hashicab" / *"hashicob"* / *"HashiCop"*	**HashiCorp**
"IEM"	**IAM**
"CC2"	**EC2**
"ST bucket" / *"SD bucket"* / *"sgbucket"* / *"Estree bucket"* / *"history bucket"* / *"mystery"*	**S3 bucket** / **in S3**
"eCase"	**EKS**
"RTS"	**RDS**
"VPN is a resource"	**VPC** (in context)
"Azio" / *"measure"*	**Azure**
"xsid"	**access key ID**
"awcline"	**AWS CLI**
"aws configur"	`aws configure`
"Kali braces" / *"callee braces"*	**curly braces**
"Bombay region, AP South one"	`ap-south-1` (Mumbai)
"Nigeria"	**N. Virginia** (`us-east-1`)
"20 22" / *"4.4.3"*	ports **22** / **443**
".tfvar file"	`.tfvars`
"plant"	**plan**
"Retreaming AWS account details"	*"retrieving AWS account details"* (credential error)
"DevWorks" / *"patch"* / *"badge a1"*	**DevOps** / **batch** / `batch-a1`
"values.liveness prop"	`livenessProbe` in a Helm values file
"ls-iPhone"	`ls -la`
"Yadri" / *"Yatri 1107"*	`yatri1107` (her brand name + number)
"DevOps heroes"	**devops-heros** (the repo's spelling)